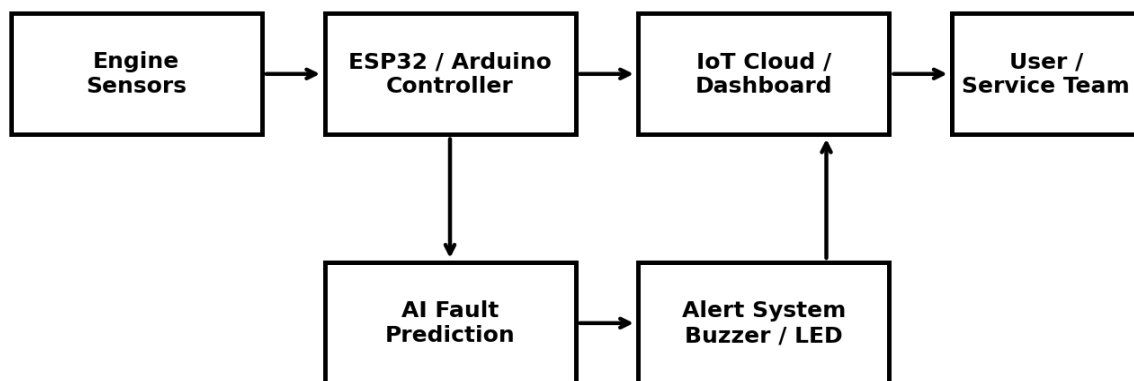


AI + IoT Based Engine Health Monitoring and Service Alert System

Prototype + Hardware Connections + Arduino/ESP32 Code + AI Concept

AI + IoT Based Engine Health Monitoring and Service Alert System



Project Objective: Monitor important engine parameters continuously, detect abnormal conditions early, predict possible faults using AI, and provide preventive service alerts before major failure occurs.

1. Problem Statement

Engine failures can happen because of overheating, abnormal vibration, low oil pressure, excessive RPM, poor maintenance, or delayed servicing. Manual inspection may not detect these problems early. Missed preventive maintenance can increase repair cost, downtime, and safety risk.

2. Proposed Solution

The proposed system combines IoT sensors, an ESP32 controller, cloud/dashboard monitoring, an AI-based fault prediction module, and a service-date/engine-hour alert system. Sensor values are collected continuously. If a value crosses a safe limit, the system generates an immediate warning. Historical sensor data can be used to train an AI model such as LSTM for early fault prediction.

3. Main Features

Feature	Purpose
Temperature Monitoring	Detect engine overheating.
Vibration Monitoring	Identify abnormal vibration and mechanical imbalance.
RPM Monitoring	Track engine speed and overspeed conditions.
Oil Pressure Monitoring	Detect low lubrication pressure.
IoT Dashboard	View live engine parameters remotely.
AI Fault Prediction	Predict abnormal/failure patterns from historical data.
Service Alert	Alert user based on service date or engine hours.
Buzzer + LED	Local warning when a critical condition occurs.

4. Prototype Hardware

Component	Example / Role
ESP32	Main IoT controller; Wi-Fi connectivity.
LM35 / Temperature Sensor	Measures engine temperature.
MPU6050	Measures vibration/acceleration.
IR / Hall Sensor	Prototype RPM pulse measurement.
Pressure Sensor	Prototype oil-pressure monitoring.
LED	Normal/warning indication.
Buzzer	Critical fault alarm.
OLED/LCD (Optional)	Local display of readings.
Power Supply	5V/regulated supply for prototype.

5. Prototype Connections

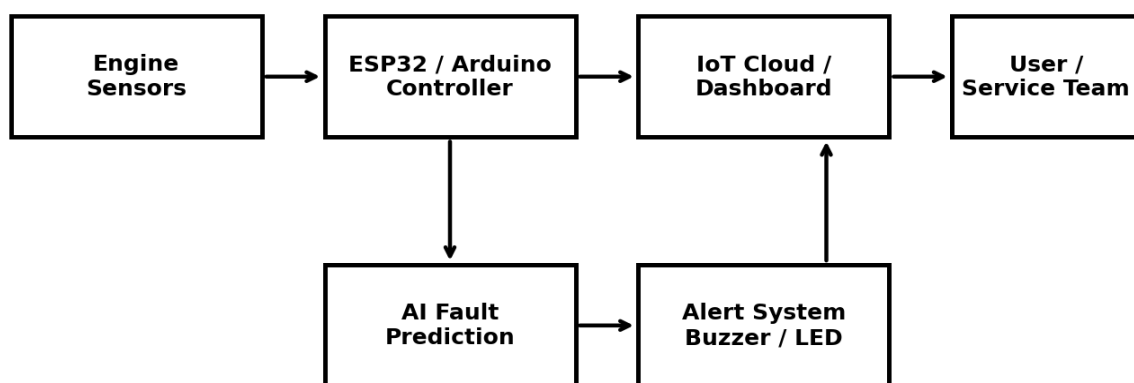
Module	ESP32 Connection	Purpose
LM35	VCC→3.3V, GND→GND, OUT→GPIO 34	Temperature
MPU6050	VCC→3.3V, GND→GND, SDA→GPIO 21, SCL→GPIO 22	Vibration
RPM Sensor	VCC→3.3V, GND→GND, OUT→GPIO 27	RPM pulses
Buzzer	Signal→GPIO 25, GND→GND	Critical alert
LED	Anode→GPIO 26 through resistor, cathode→GND	Warning status

Prototype note: The exact sensor used for a real engine must be selected according to engine type, temperature range, pressure range, electrical isolation, vibration level, and safety requirements. The connections above are intended for a low-voltage demonstration prototype.

6. Working Flow

1. Sensors collect engine temperature, vibration, RPM and oil-pressure data.
2. ESP32 reads and filters the sensor values.
3. Threshold rules identify immediate abnormal conditions.
4. ESP32 sends readings to an IoT dashboard through Wi-Fi.
5. Historical readings are stored for AI model training.
6. AI model predicts the probability of abnormal/failure conditions.
7. If a critical condition occurs, buzzer/LED gives a local warning.
8. Service alert is generated when the service date or engine-hour limit is reached.

AI + IoT Based Engine Health Monitoring and Service Alert System



7. Suggested Prototype Dashboard

Parameter	Normal Example	Warning Example
Temperature	70–90 °C	> 100 °C
Vibration	Low/stable	Sudden increase
RPM	Within rated range	Overspeed
Oil Pressure	Within engine specification	Below minimum
Service	Not due	Due soon / overdue
AI Health Score	Healthy	Attention / Critical

8. ESP32 Prototype Code

Install the **MPU6050** Arduino library before compiling. Select an ESP32 board in Arduino IDE. For the first demo, open Serial Monitor at 115200 baud and vary the sensor inputs to demonstrate warning generation.

```
/*
  AI + IoT Based Engine Health Monitoring
  Prototype Code - ESP32 + LM35 + MPU6050 + RPM sensor
  + LED + Buzzer

  NOTE:
  - This is a demonstration prototype.
  - Calibrate sensor thresholds for the actual engine.
  - IoT cloud upload can be added using ThingSpeak,
    Blynk, MQTT, Firebase, etc.
*/

#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

const int TEMP_PIN = 34;    // LM35 analog output
const int RPM_PIN  = 27;    // RPM pulse input
const int LED_PIN  = 26;
const int BUZZER_PIN = 25;

volatile unsigned long pulseCount = 0;
unsigned long lastMillis = 0;

float temperatureC = 0;
float vibration = 0;
float rpm = 0;

// Demo thresholds - change for your engine
const float TEMP_LIMIT = 100.0;
const float VIBRATION_LIMIT = 1.50;
const float RPM_LIMIT = 3000.0;

void IRAM_ATTR rpmPulse() {
  pulseCount++;
}

void setup() {
  Serial.begin(115200);

  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(RPM_PIN, INPUT_PULLUP);

  attachInterrupt(digitalPinToInterrupt(RPM_PIN),
    rpmPulse, RISING);

  Wire.begin(21, 22);
  mpu.initialize();

  if (mpu.testConnection()) {
    Serial.println("MPU6050 connected");
  } else {
    Serial.println("MPU6050 connection failed");
  }

  lastMillis = millis();
}

void loop() {

  // ----- Temperature -----
  int raw = analogRead(TEMP_PIN);
  float voltage = (raw / 4095.0) * 3.3;

  // LM35: 10 mV per degree C
  temperatureC = voltage * 100.0;

  // ----- Vibration -----
  int16_t ax, ay, az;
```

```

mpu.getAcceleration(&ax, &ay, &az);

float axg = ax / 16384.0;
float ayg = ay / 16384.0;
float azg = az / 16384.0;

// Simple vibration magnitude
vibration = sqrt(axg*axg + ayg*ayg + azg*azg);

// ----- RPM -----
if (millis() - lastMillis >= 1000) {
    noInterrupts();
    unsigned long pulses = pulseCount;
    pulseCount = 0;
    interrupts();

    // Assumption: 1 pulse = 1 revolution
    rpm = pulses * 60.0;

    lastMillis = millis();
}

// ----- Rule-based health check -----
bool critical = false;

if (temperatureC > TEMP_LIMIT) critical = true;
if (vibration > VIBRATION_LIMIT) critical = true;
if (rpm > RPM_LIMIT) critical = true;

// ----- Alert -----
if (critical) {
    digitalWrite(LED_PIN, HIGH);
    digitalWrite(BUZZER_PIN, HIGH);
} else {
    digitalWrite(LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);
}

// ----- Serial dashboard -----
Serial.println("----- ENGINE STATUS -----");
Serial.print("Temperature: ");
Serial.print(temperatureC);
Serial.println(" C");

Serial.print("Vibration: ");
Serial.println(vibration);

Serial.print("RPM: ");
Serial.println(rpm);

if (critical) {
    Serial.println("STATUS: CRITICAL - CHECK ENGINE");
} else {
    Serial.println("STATUS: HEALTHY");
}

Serial.println();

delay(1000);
}

```

9. AI Fault Prediction Module

For the advanced version, collect labelled engine data containing temperature, vibration, RPM, oil pressure and a fault/healthy label. A Random Forest model is suitable for a simple proof-of-concept. For large time-series datasets, an LSTM model can learn patterns across time and provide earlier fault prediction.

```
# Simple AI training concept (Python)

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Example CSV columns:
# temperature, vibration, rpm, oil_pressure, fault
data = pd.read_csv("engine_data.csv")

X = data[["temperature", "vibration", "rpm", "oil_pressure"]]
y = data["fault"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

prediction = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, prediction))

# Predict a new engine condition
new_data = [[92, 1.2, 2100, 2.8]]
result = model.predict(new_data)

if result[0] == 1:
    print("AI ALERT: Possible abnormal condition")
else:
    print("AI STATUS: Normal")
```

10. Service-Date / Engine-Hour Alert

The service feature can use either a real calendar date or accumulated engine operating hours. A real-time clock (RTC) can track date-based service intervals. Engine-hour based maintenance is useful when the engine's running time is more important than calendar time.

```
// Service alert concept
// Use engine-hour counter or a real RTC/date module.
// Example logic:

const unsigned long SERVICE_HOURS = 500;

float engineHours = 495.5;

if (engineHours >= SERVICE_HOURS) {
    Serial.println("SERVICE ALERT: Maintenance Due!");
}

// In a complete IoT version:
// ESP32 -> Cloud -> Dashboard/mobile notification
// Store last service date and next service date.
// Notify when due date is near or engine hours exceed limit.
```

11. Prototype Demonstration Procedure

Step	Demo Action	Expected Result
1	Power the ESP32 and sensors.	System starts.
2	Open Serial Monitor.	Live values appear.
3	Warm the temperature sensor safely.	Temperature reading rises.
4	Move/vibrate the MPU6050 module.	Vibration value changes.
5	Generate RPM pulses.	RPM value changes.
6	Cross a programmed limit.	LED + buzzer activate.
7	Show dashboard concept.	Remote monitoring explained.
8	Set service hours/date as due.	Service alert appears.

12. Expected Output

The prototype displays live engine parameters, identifies abnormal values, activates a local warning, and provides the foundation for remote IoT monitoring. With historical data, the AI module can estimate the likelihood of an upcoming fault. The service alert reduces the chance of missed preventive maintenance.

13. Future Enhancements

- Cloud dashboard with live graphs and engine history
- Mobile notification through Blynk/Firebase/MQTT
- LSTM-based time-series predictive maintenance
- Engine-specific fault classification
- GPS/location tagging for fleet monitoring
- Automatic service-history records
- Multiple-engine fleet dashboard
- Secure industrial sensor interfaces and electrical isolation

14. Key Judge/Viva Points

Why IoT? To monitor the engine remotely and store historical data.

Why AI? Thresholds detect known abnormal values; AI can learn patterns and predict faults earlier.

Why service alerts? Preventive maintenance reduces the chance of breakdown caused by missed servicing.

What is unique? The system combines real-time sensing, remote monitoring, predictive analytics and maintenance scheduling in one low-cost prototype.

Safety: This prototype is for academic demonstration. Do not connect hobby sensors directly to high-voltage, high-temperature, pressurized, or safety-critical engine systems without appropriate industrial sensors, signal conditioning, isolation, protection and professional validation.